CSA08 - Python Programming

ASSIGNMENT 3

CO3

Lists, Tuples, Dictionaries and Sets

Total Marks: 50 | Individual Assignment

Solutions for Q1-Q7

Student Name: PROBIR SAHA SHOHOM

Register Number: 1911260319

GitHub Repository Link:
https://github.com/Probir127/CSA08-1911260319-Assignment3

Student and scope

Student Name: PROBIR SAHA SHOHOM. Register Number: 1911260319. Course: CSA08 - Python Programming. GitHub Repository Link: https://github.com/Probir127/CSA08-1911260319-Assignment3

This report solves Q1-Q7 of Assignment 3 (CO3), total 50 marks, covering lists, tuples, dictionaries, sets, comprehensions, nested structures and contact merging. Each question includes implementation, analysis, experiments and a defended choice. Completed TODO programs are supplied separately with the original runnable structure preserved. Full Python source is included in the appendix and repository; generators make every synthetic dataset reproducible.

The programs were executed with assertions enabled using 3.15.2 (v3.15.2:5e58eaad7c5, Oct 25 2026, 14:19:34) [Clang 21.0.0 (clang-2100.1.1.101)]. Only standard-library modules are needed. Recorded elapsed seconds and memory values are observations for this environment, not universal constants. Except for explicitly repeated trials, a timing is one recorded run. Counts and known input sizes supplement noisy short timings. No real railway schedule, private contacts, patient data or farmer complaints were used.

Prepared with Codex assistance. The student must understand and review the work, run the code and submit the repository URL to the faculty. Task commits record this preparation session, not a week of independent progressive work. Output PNGs are labelled images rendered from actual captured stdout; they are not terminal screenshots. Faculty-required terminal screenshots and final course submission remain student actions. No faculty invitation is made because a faculty GitHub identity was not supplied.

Q1 - Lists and slicing: rainfall (6 marks)

Design and implementation

A list preserves chronological order, supports integer indexing and slices, and permits later correction. The input is one nonnegative integer-mm reading per day. Generated datasets use random.Random with seed 1911260319 at 365, 36,500 and 365,000 days; zeros represent dry days. Integer measurements deliberately avoid floating-point running-sum drift while still producing fractional averages by division. Real fractional measurements can use an agreed fixed unit such as tenths of a millimetre or a documented tolerance.

Weekly totals use `[sum(readings[i:i+7]) for i in range(0, len(readings), 7)]`. This includes a final partial week: 365 days give 52 complete weeks and a one-day week 53. Wettest/driest results report all tied week positions using one-based numbering. A one-day subtotal is not directly comparable with a full seven-day total; the report retains it explicitly rather than silently dropping day 365. An operational comparison could instead label/exclude partial weeks or compare per-day means. Dry days are one-based positions whose reading is zero.

For the hand example `[0,1,2,3,4,5,6,7,0,0,0,0,0,0]`, weekly totals are [21,7]; week 1 is wettest and week 2 driest. Its dry days are 1 and 9-14. The first moving average is 21/7=3 and the second is 28/7=4. The naive, running-sum and deque results agree exactly. Empty inputs produce no weeks/averages; a list shorter than seven days produces no full-window average; seven zeroes produce one average of zero.

For the generated year: wettest week positions [45] with total 408 mm; driest week positions [53] with total 0 mm. There are 78 zero-rain days.

The complete list of 53 totals and every dry day is preserved in Q1/year_summary.json and output.txt, avoiding an unreadable 365-value table in the report.

Slices, costs and pseudocode

`readings[7:14]` returns days 8-14, `readings[-7:]` the final seven available readings, and `readings[::7]` days 1,8,15,... . Slice stops are excluded, and a slice creates a new list of references. A slice of k elements costs O(k) time/space. Append is amortized O(1), insert at position i shifts O(n-i) references, pop at the end is O(1), pop near the front is O(n), and sort is O(n log n) worst-case with up to O(n) temporary reference storage in CPython. Indexed access is O(1).

Running-sum pseudocode:

```
ALGORITHM MOVING_AVERAGE(readings, k)
BEGIN
    REQUIRE k > 0
    IF LENGTH(readings) < k THEN RETURN empty list END IF
    total <- 0
    FOR i <- 0 TO k-1 DO total <- total + readings[i] END FOR
    averages <- [total / k]
    FOR i <- k TO LENGTH(readings)-1 DO
        total <- total - readings[i-k]
        total <- total + readings[i]
        APPEND total / k TO averages
    END FOR
    RETURN averages
END
```

This uses conventional structured pseudocode. The detailed institutional SIMATS template was not included in the supplied PDF; exact compliance with an unavailable formatting template is not claimed.

Derivation and measured comparison

Let n>=7 and w=n-6 full windows. The instrumented naive implementation initializes total=0 and executes seven additions per window: 7(n-6). A version assigning the first value initially would count six additions instead, but that is not the executed program. The running version uses seven initial additions plus n-7 further additions, for n additions and n-7 subtractions, or 2n-7 arithmetic updates. Both perform w divisions. For n<7, both return immediately with zero additions.

Days	Windows	Naive additions	Running adds / subtracts	Naive / running time (ms)
365	359	2513	365 / 358	0.0417 / 0.0183
36500	36494	255458	36500 / 36493	4.2015 / 1.5234
365000	364994	2554958	365000 / 364993	42.4985 / 16.2215

The results are equal in every tested dataset. Timings include the explicit count increments and output-list creation for each algorithm. The running approach reduces repeated arithmetic and slice allocation; time ratios need not equal operation-count ratios. For a fixed window size 7, both are O(n) asymptotically; for general window size k, naive is O(nk) and running sum O(n). Both retain O(n) returned averages in these functions. Running sum needs O(1) extra accumulator space beyond retained input and result; naive needs O(k) transient slice space.

Late readings and alternatives

A late insertion into a chronological list costs O(n) in the worst case. A list remains a good choice when arrivals are mostly ordered and corrections are uncommon. When missing dates are already represented by slots, replacing a reading is O(1) and preferable to inserting and shifting all later dates. A dictionary keyed by date supports average O(1) insertion/lookup but chronological traversal requires sorted dates, generally O(n log n), unless a complete ordered calendar is maintained. Missing days need an explicit policy; they must not automatically become zero rainfall.

The supplied deque_average uses deque(maxlen=7), subtracts the outgoing first value before append evicts it, and maintains a running total. It can consume a stream using O(7) window storage and O(1) arithmetic per new day. Re-summing the deque each time instead costs O(7n), or O(nk) generally. The demonstrated function still returns an O(n) output list; yielding averages would avoid retaining it. A dictionary stores O(n) date/value entries and has greater per-entry overhead than a dense list but makes date identity explicit.

Q2 - Tuples, mutability and copying (7 marks)

Completed exercise and identity evidence

The correct TODO is `good = [[0] * cols for _ in range(rows)]`. Each comprehension iteration creates a distinct row. The supplied program's exact results are:

```
bad : [[0, 7, 0, 0], [0, 7, 0, 0], [0, 7, 0, 0]]
good: [[0, 7, 0, 0], [0, 0, 0, 0], [0, 0, 0, 0]]
True False
T. Nagar
```

Memory diagram for the requested 3x3 illustration:

```
bad = [[0]*3]*3                 good = [[0]*3 for _ in range(3)]
outer[0] --+                    outer[0] ---> row A [0,0,0]
outer[1] --+--> one row [0,0,0]  outer[1] ---> row B [0,0,0]
outer[2] --+                    outer[2] ---> row C [0,0,0]
```

Q2/copying.py prints actual row ids. All bad rows have the same id and `bad[0] is bad[1]` is True. Correct rows have distinct ids and the corresponding is test is False. Numeric ids are process-specific, so their values are not expected to repeat on another run. The complete-code grid is 3x4 as specified; the memory diagram uses the separately requested 3x3 form.

Copying, arguments and tuple hashability

With `original=[[1],[2]]`, alias=original creates no copy; `original[:]` and copy.copy create new outer lists sharing inner rows; copy.deepcopy recursively copies the rows. After original[0].append(7), alias, slice and shallow show [[1,7],[2]], while deep remains [[1],[2]]. Shallow copying is enough for a flat list of immutable values or when nested sharing is intentional; use deep copying when mutable nested state must be independent. Deep copying can be expensive, and references intended to stay shared should not be duplicated blindly.

A function receives a reference to the same list object. Calling append inside it changes that object, so the caller sees the change. Rebinding the parameter to a new local list would not rebind the caller's name. The executed mutate example changes the caller's empty list to [9].

Coordinates `(latitude, longitude)` are meaningful fixed-length records and can be unpacked in `for (lat, lon), stop in stops.items()`. A tuple is hashable only if all its members are hashable. The float-coordinate tuple keys work, while `[13.0418,80.2341]` raises TypeError: unhashable type: 'list'. Lists are mutable and unhashable. A tuple containing a list is also unhashable; tuple immutability protects its slots, not objects reachable through those slots. `t=([1,2],'stop'); t[0].append(3)` succeeds, giving ([1,2,3],'stop').

GPS float keys require exact float equality. For approximate measured positions, use stable stop IDs or a deliberate coordinate normalization rule rather than assuming nearby GPS readings are identical keys.

Memory and timing evidence

Container (1,000,000 values)	Shallow bytes	Creation runs (ms)	Mean 10,000-index time (ms)
list	8000056	5.7803 / 9.3456 / 10.0767	0.488264
tuple	8000048	6.7961 / 10.4419 / 10.5827	0.488750

Sizes from sys.getsizeof are shallow: they include the container and reference slots, not all one million integer objects. The small byte difference here is specific to this Python build and exact allocation pattern; lists are not always vastly larger. Creation runs use list(range(n)) and tuple(range(n)), including integer production; reassignments can include release of previous results. The indexing experiment uses the same 10,000 generated positions and a checksum, with three runs per container. This evidence does not support an unconditional claim that tuples are faster. Both have O(1) indexed access and O(n) construction/storage. Choose tuples for fixed records and protection against slot replacement, and lists for changing collections.

Sparse-grid alternative

```
passengers = {}                  # (row, column) -> nonzero passenger count
key = (1, 2)
passengers[key] = passengers.get(key, 0) + 1
count = passengers.get((0, 0), 0)  # absent means zero
```

Dense row/column lookup is O(1); dictionary lookup is average O(1) with hash-table overhead. A dense grid occupies O(rows*cols) slots, even for empty zones; a sparse dictionary occupies O(z) entries for z nonempty zones. Remove entries whose count returns to zero, and validate coordinate bounds separately. Sparse representation is useful for mostly empty large grids; a dense grid is simpler and more compact per occupied cell when most cells are used.

Q3 - Farmer helpline dictionaries (7 marks)

Counting, grouping and completed code

The completed TODO counts non-stop words using `counts[word] = counts.get(word,0)+1`. Top words use `sorted(counts.items(), key=lambda pair: (-pair[1],pair[0]))[:k]`. The exact supplied result is `[('seeds',3), ('price',2), ('received',2)]`. Price comes before received because the second sort key is alphabetical, not because of its insertion position. The required fixture is already lowercase and whitespace-separated, so its original format is preserved.

The additional complaints.py handles uppercase/lowercase and punctuation by converting letters to lowercase and other characters to separators. This is a simple word policy: digits and apostrophes are not retained within tokens. Stop words use a set for average constant-time membership. Empty text contributes no tokens. Tested mixed text gives {'seeds':3,'delayed':1,'price':1}. Grouping uses `district.setdefault(name,[]).append(record)` and an equivalent crop dictionary; district counts are Chennai=2, Madurai=1, and the rice crop has two complaints. The original complaint records are retained in grouped lists.

A dictionary hashes a key to select a table region and resolves collisions with key equality. With a suitable hash distribution and controlled load factor, lookup/insertion is average O(1), rather than scanning every key. This is not a worst-case guarantee: collisions and the cost to hash/compare long strings matter. Python 3.7+ dictionaries preserve insertion order; this is not automatic alphabetical or frequency ordering.

keys(), values() and items() return dynamic views of keys, values and (key,value) pairs. The views reflect later dictionary changes and normally occupy O(1) metadata, while iterating them takes O(d). `list(counts.items())` makes a snapshot. Changing dictionary size during iteration can raise RuntimeError.

Experiments and decision

The generator repeats four synthetic complaint templates. Normalization and tokenization happen once before the counting-only timers; this makes the compared counting inputs identical. There are 11 distinct retained words at each size. Counter, get, setdefault and list.count results are asserted identical.

Complaints	Words / distinct	get / setdefault (ms)	list.count / Counter (ms)	W*d model
1000	3750 / 11	0.0893 / 0.1214	0.1841 / 0.0562	41250
10000	37500 / 11	1.0303 / 1.4121	1.8231 / 0.6667	412500
100000	375000 / 11	9.8486 / 13.7756	18.5726 / 6.9977	4125000

The dictionary and Counter loops take expected O(W) time for W words, and O(d) count storage. `list.count` scans W words for each of d distinct words, giving O(Wd) equality checks; building its distinct-word set adds average O(W). The retained token list uses O(W) extra space in this benchmark. The table's W*d is a derived count model, not an instrumented number of C-level comparisons. Because d is fixed at 11, these measurements need not show quadratic growth in complaint count. If vocabulary grows proportionally to W, the alternative can approach O(W^2). Sorting all d words for top k costs O(d log d) time and O(d) storage; slicing to k does not remove the full sort cost.

At 100,000 complaints, Counter was fastest in this measured dataset. Deploy Counter for a straightforward count or a get loop when the assignment requires explicit accumulation/custom logic; use setdefault naturally for grouping. Process complaints as a stream when the raw token list is unnecessary, while retaining original records in a suitable store if an audit or crop-specific report is required. Average O(1) hash operations do not make the full analysis O(1).

Q4 - Sets and elective clashes (7 marks)

Hand case and completed exercise

Let A be AI registrations, B IoT registrations and U the set of all students. Use `A & B` for clashes, `A ^ B` for exactly one and `U - (A | B)` for neither. Union gives every registered student; A-B gives AI-only. The executed required output is:

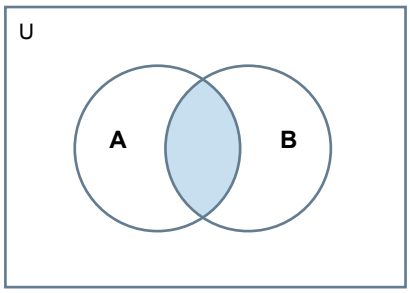
```
clash      : ['192311004', '192311009']
exactly one: ['192311001', '192311002', '192311007', '192311010']
neither    : ['192311003', '192311005', '192311006', '192311008']
```

The method forms `A.intersection(B)`, `A.symmetric_difference(B)`, `U.difference(A.union(B))` are executed and equal to the operator results. All registration sets are assumed subsets of U; unexpected IDs should be validated rather than reported as enrolled students outside the official roster.

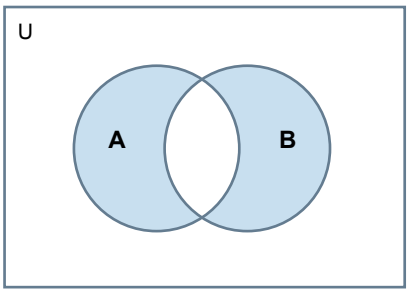
Venn diagrams

Blue areas are selected; outlined circles are electives inside the rectangular universe. The first three diagrams show the three required reports, and the last two illustrate union and difference.

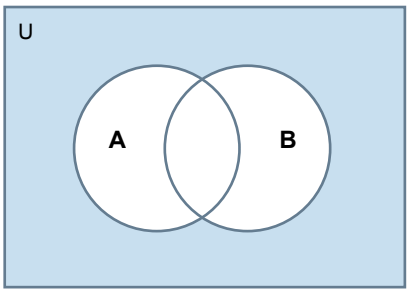
A & B



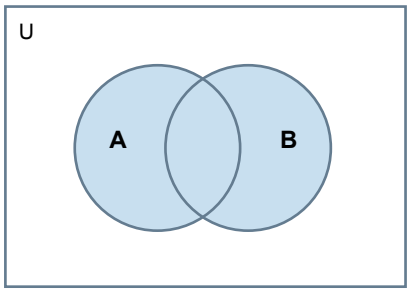
A ^ B



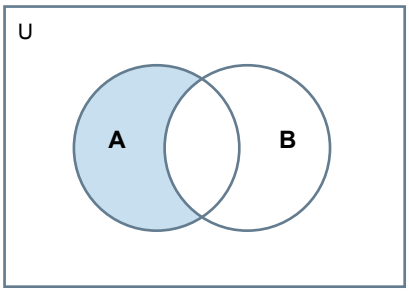
U - (A | B)



A | B



A - B



Sets store one occurrence per equal hashable key, so converting ['x','x','y'] produces {'x','y'}. Lists cannot be members because they are unhashable; tuples can be members only when every contained element is hashable. Sets do not provide a meaningful enrollment order. Producing a sorted report adds O(r log r) work for r reported IDs.

Measured membership and intersection

IDs	List time: 20 probes (ms)	Set time: 20 probes (ms)	Set shallow bytes
10000	0.780959	0.000959	524504
100000	7.860750	0.001000	4194520
1000000	79.233541	0.004541	33554648

Each membership timer performs 20 probes: ten for the last list item and ten for a missing item. Thus list scans traverse approximately n entries per probe, exposing the O(n) case. Set construction is excluded from these lookup timings and hashes have already been computed during construction. Set shallow sizes exclude the referenced ID strings. Microsecond set timings should not be treated as precise throughput guarantees.

n=m	Nested comparisons	Common IDs	Nested / sets / sort-walk (ms)
100	10000	50	0.1017 / 0.0088 / 0.0107
1000	1000000	500	9.7234 / 0.0211 / 0.0437
3000	9000000	1500	90.5885 / 0.0516 / 0.1461

The nested comparison counts are explicitly instrumented n*m. Set timing here includes constructing both sets from the input lists as well as intersection. Sort/walk timing includes sorting copies; these generated inputs are already ordered, so Python's adaptive sort benefits and does not demonstrate its worst case. All methods produce the same common IDs. Empty electives, one empty elective and identical electives are tested; symmetric difference of identical electives is empty and neither then contains U-A.

Nested loops take O(nm) time; output de-duplication uses a set. Sorting and walking takes O(n log n + m log m) worst-case sorting plus O(n+m) traversal, and O(n+m) copy storage in this implementation. Hash-based construction/intersection takes expected O(n+m) total time with O(n+m) set storage; intersection of already built sets typically iterates the smaller set. Convert a list to a set when repeated membership/uniqueness or set algebra justifies the average O(n) conversion and O(n) storage. Conversion discards multiplicity and meaningful input ordering, so keep the original list if these are needed.

Q5 - Comprehensions and sensor readings (6 marks)

Cleaning policy and three implementations

All examples are synthetic numerical readings. Values -50..50 inclusive are valid sensor measurements. Among valid readings, temperatures outside 2..8 inclusive raise cold-storage alarms. Invalid sensor readings are excluded from temperature alarms and should instead be flagged as sensor faults in a real system. Minute numbers refer to original positions starting at 1; removing invalid values must not renumber later alarms.

For [-51,-50,2,8,9,50,51], the hand-checked valid Celsius list is [-50,2,8,9,50], Fahrenheit is [-58.0,35.6,46.4,48.2,122.0], and alarm minutes are [2,5,6]. The loop/append, list-comprehension and map/filter/lambda pipelines produce identical tuples of lists. Empty input, all-valid, all-invalid and exact boundary cases pass. Their input contract is numeric readings; real files containing text, missing values or nonfinite numbers need an additional parsing/validation stage.

General comprehension form is `[expression for item in iterable if condition]`. The source also executes:

```
by_minute = {i: c for i, c in enumerate(readings, 1) if -50 <= c <= 50}
distinct_valid = {c for c in readings if -50 <= c <= 50}
total_valid = sum(c for c in readings if -50 <= c <= 50)
```

The dictionary retains original minute identity; the set keeps unique temperatures and loses count/order. Comprehensions are useful for one clear mapping/filtering rule. Deep nesting, multiple side effects or elaborate error

handling are better expressed by a loop. map/filter in Python are lazy iterators, but this program converts their results to lists because the required output is three reusable reports.

Timings and memory

Readings	Loop / comprehension / functional (ms)	Valid values	List / generator bytes	List / generator sum (ms)
10000	0.6190 / 0.7106 / 1.2863	8356	67224 / 224	0.0423 / 0.2099
100000	6.0009 / 6.9389 / 12.9788	83416	711960 / 224	0.3838 / 2.0411
1000000	62.4030 / 69.4737 / 132.1657	834675	7509944 / 224	4.1038 / 20.6397

The explicit loop was fastest for this combined workload in all three observed sizes. It performs filtering, conversion and alarm detection in one traversal; the comprehension version traverses the original readings again for alarms. This is a comparison of the implemented pipelines, not proof that the syntax of a loop is universally faster than a comprehension. The map/filter version adds lambda calls and still materializes all reports. All have O(n) time and O(n) retained result space.

A list comprehension eagerly allocates all selected references. A generator expression stores iteration state and produces values on demand. sys.getsizeof is shallow: the generator's fixed object size excludes the input list retained by its reference, and the list's size excludes element objects. Both sum variants visit all selected values; the list-only sum timer excludes the earlier cost of constructing/filtering the list, whereas the generator sum timer includes filtering during consumption. They are not an end-to-end speed comparison. In a one-pass sum, a generator can avoid an extra O(n) materialized list, using O(1) additional generator state beyond the existing input and numeric accumulator. It cannot be re-used after exhaustion without rebuilding it. If the application needs Celsius, Fahrenheit and alarm lists repeatedly, materialization is justified.

Q6 - Nested railway timetable (7 marks)

Model and small query

The fixture contains five fictional trains. Train numbers are dictionary keys; each value is a dictionary with name, ordered stop list and running-day set. Each stop is a fixed `(station,arrival,departure)` tuple. Time strings are zero-padded HH:MM on the same service day, so lexicographic order matches clock order. The model does not yet interpret overnight date offsets, time zones or live timetable changes, and is not travel advice.

```
timetable
  'T101' -> dictionary
      name  -> 'Village Link'
      stops -> list
          0 -> ('North',   '08:00', '08:05')
          1 -> ('Central', '09:00', '09:05')
          2 -> ('South',   '10:00', '10:05')
      days  -> {'Mon', 'Wed', 'Fri'}
  'T102' -> another train dictionary
  ... five trains in the fixture
```

The third stop's departure is `timetable['T101']['stops'][2][2]`, giving 10:05. The outer dictionary supports train-ID lookup; the inner dictionary names fields; the stop list preserves route order; tuples preserve stop-record structure; the day set provides direct membership and uniqueness. Nested mutable containers remain mutable even when some records are tuples.

Central on Monday yields the independently expected order T104 (07:05), T102 (08:35), T101 (09:05). T103 is excluded because it runs only on Saturday/Sunday. An unknown station returns an empty list. Queries sort using `sorted(matches,key=lambda row:(row[3],row[0]))`, with departure first and train ID as a deterministic tie-breaker. Station/day spellings are exact in this teaching interface.

Index design, work and measurements

The index maps station to a list of `(train_id,arrival,departure)` occurrences. Storing the stop details directly is important: an index holding only train IDs would still need to search each matching train's stops. Build it once, then filter station candidates by running day and sort matches.

If S is total stops, m occurrences are at the station and h are returned for that day, an unindexed query costs O(S + h log h), while the indexed query costs expected O(m + h log h). Building the index takes expected O(S) time and O(S) additional references/record tuples. A station unknown to the index is an average O(1) empty lookup. A station served by nearly every train gives less benefit. Repeated calls require updating/rebuilding the index when the base timetable changes.

Trains / stops	Candidates	Scan / indexed / flat query (ms)	Index build (ms)	Index container bytes
100 / 1000	10	0.0130 / 0.0011 / 0.0183	0.0811	93728
1000 / 10000	100	0.1338 / 0.0144 / 0.1958	0.5615	815328
10000 / 100000	1000	1.4741 / 0.2095 / 1.9843	9.7924	8088928

Each synthetic train has ten unique station stops, chosen from 100 station labels; the target has about 1% of all stop occurrences. Query times are averages over 20 equivalent queries, excluding generation and index construction. Every scan/index/flat result is asserted equal. The extra-memory estimate sums newly allocated index dictionary, list and tuple containers, excluding shared strings, the original timetable and interpreter allocator overhead. It is a container-overhead estimate, not a whole-process memory measurement.

The flat alternative stores `(train,name,station,arrival,departure,days)` rows and scans all S rows per query, then sorts its matches: O(S+h log h), using O(S) row storage. A flat list can be simple for CSV export but duplicates references to train fields. The station index pays a one-time construction and storage cost to avoid repeated unrelated-stop scans. For occasional tiny queries a scan may be adequate; for repeated enquiry-counter queries the measured index is worth retaining.

Q7 - Contact merging from first principles (10 marks)

Stage 1: interpret and measure requirements (2 marks)

Input is a list of dictionaries with optional name, numbers and emails fields; number/email fields are collections. Output is one record per connected person group, retaining all distinct normalized numbers, emails, name aliases, source row IDs and rejected phone strings. Synthetic records are used throughout.

Identity rule: two records belong together only if they share a canonical phone number, directly or through a chain of records. Equal names alone do not establish identity, and email-only records are preserved independently under this chosen policy. Measurable requirements are: retain every input source ID once, merge formatting variants, de-duplicate fields, preserve all numbers/emails, keep different-number same-name people separate, handle missing fields, and merge bridge records joining previously separate groups.

Phone assumptions are explicit: unprefixed 10-digit numbers are Indian numbers; +91 and 0091 variants become +91 followed by ten digits; benign spaces, hyphens, parentheses and dots are removed; malformed lengths/characters are flagged. This is a stated dataset policy rather than a general international telephone-number validator. Names are case-folded, punctuation/extra whitespace normalized and a leading Dr/Mr/Mrs/Ms title removed for alias presentation only. Emails are stripped and case-folded as an explicit practical policy; full RFC email validation and case-sensitive local parts are outside this model.

Stage 2: design and pseudocode (3 marks)

A dictionary `owner[number]` associates each normalized number with a representative input row. Union-find tracks connected components, with path compression and union by size. Numbers, emails and aliases are sets so duplicates collapse. Crucially, encountering one record containing two previously unrelated numbers must union both groups; keeping only one arbitrary number as a dictionary key would lose such transitive identity links.

```
ALGORITHM MERGE_CONTACTS(records)
BEGIN
    normalized <- normalize each input record; retain rejected numbers
    MAKE one component for each input row
    owner <- empty dictionary
    FOR EACH row i IN normalized DO
        FOR EACH number IN row.numbers DO
            IF number is in owner THEN
                UNION component(i) with component(owner[number])
            ELSE
                owner[number] <- i
            END IF
        END FOR
    END FOR
    groups <- empty dictionary
    FOR EACH normalized row i DO
        root <- FIND component(i)
        UNION its name, number, email and source-ID sets into groups[root]
        APPEND its rejected-number evidence into groups[root]
    END FOR
    RETURN deterministic reports for the groups
END
```

Records without valid numbers remain separate components rather than being accidentally merged under an empty key. Final groups follow first-occurrence input order; individual set fields are sorted for reproducible display. The implementation creates no new fabricated contact details.

Stage 3: implement, hand-check and measure (3 marks)

The six-row hand fixture predicts four groups. Rows 0,1,2 merge through shared 9840012345 and 9876543210: aliases include ravi, ravi kumar and r kumar; the result retains both +91 numbers and the two email addresses. Row 3 has the same normalized name but number 9123456789, so it remains a separate person. Row 4 has no phone and is retained alone. Row 5 has an invalid number and empty name, remains alone and preserves its rejected string for correction. Assertions check these source-ID groups exactly. A separate bridge test joins records with phone A, phone B, and a third record containing both A and B. Empty input returns no groups. The every-pair reference and dictionary implementation agree on the hand fixture and on all 1,000 generated records.

Contacts	Groups	Phone occurrences	Dictionary time (ms)	Required pair tests	Full pairwise time
1000	200	1000	2.4422	499500	30.3641 ms
100000	20000	100000	403.3412	4999950000	Not completed; timed trial below

The large generator makes five formatted entries per synthetic person. Dictionary timing includes normalization, component grouping and deterministic output materialization; data generation is excluded. It correctly returns 200 groups from 1,000 records and 20,000 groups from 100,000 records, retaining every source ID.

The 100,000-contact pairwise trial reported **TIMEOUT_3_SECONDS**. Required comparisons: 4,999,950,000. Last actual checkpoint: 30,800,000 comparisons at worker elapsed 2.9345 s; parent elapsed 3.0038 s. The full-run completion time was not measured. See Q7/large_trial.json for raw evidence.

The every-pair comparison at 100,000 contacts is deliberately bounded, not presented as a completed benchmark. Its checkpoint is a lower bound on actual completed comparisons because progress is written only every 100,000 pairs. Parent elapsed time includes process start and termination; worker elapsed time starts after input generation and includes normalization, comparisons and periodic progress writes. The 1,000-record baseline has no progress callback, so the guarded trial also has some extra measurement overhead. No exact full-run duration or speedup at 100,000 is inferred from the timeout. This is a disclosed limit of the experimental evidence.

Stage 4: defend the design and limitations (2 marks)

For N contacts and P phone occurrences with bounded field sizes, hash dictionary work is expected O(P); disjoint-set operations add a near-constant amortized inverse-Ackermann factor, O(P alpha(N)), plus O(N) grouping. This is commonly summarized as near-linear O(N) for a bounded number of numbers per contact, whereas checking every pair needs N(N-1)/2 tests, O(N^2), even when most pairs are unrelated. If a contact contains many numbers, their set-intersection/normalization costs must also be counted; complexity is better expressed in total field size than record count alone.

Owner, union-find and merged fields occupy O(N+P+E) space for E email/name content references, plus normalized input/output. Sorting report fields adds the sum of O(g log g) for each field-set size g; a single giant merged group can therefore add O(N log N) output-formatting work. This cost is separate from the near-linear grouping core. Every-pair processing still needs its normalized records, component state and output, O(N+P+E), although it avoids the number-owner dictionary.

The measured 1,000-record result validates equality and shows the dictionary design faster; the large dictionary run completed, while the guarded pairwise run did not finish within three seconds. This supports choosing the dictionary design, without pretending the timeout is an exact completed runtime.

Remaining limitation: a shared/reassigned household phone can merge two real people who are not the same person; conversely, different numbers and no shared record can leave one person split across groups. Names alone cannot safely resolve this. Ambiguous groups should be reviewed or matched with additional trusted identifiers. The same-name test demonstrates the deliberate avoidance of name-only false merges, not a perfect real-world identity resolver. Changes to country or email policies require re-normalization and re-testing.

Verification and submission

Every question has executable source and captured output; Q2-Q4 also retain the required completed exercises. Measurements, hand fixtures and generator logic accompany the report. `python3 tools/run_all.py` runs all examples and assertions, then writes verification.json. It treats the specifically designed large pairwise timeout as a recorded experimental outcome, not as a failed correctness test. Avoid Python's -O option, which disables assertions.

Before submission, review the code and assumptions, capture any explicitly required terminal screenshots and submit the GitHub URL to the faculty. Raw measurements and operation counts remain available to explain or reproduce the results during verification. Re-running changes timings; report tables should be regenerated from the corresponding new measurements before claiming they describe that later run.

Appendix - Complete Python source

Each listing is labeled with its task folder and filename. Save related files in the shown folders. The companion ZIP also includes sample inputs and captured execution output. Long source lines are visually wrapped in this PDF; use the files in the ZIP when running the programs.

Q1/rainfall.py

```
"""Weekly summaries and instrumented seven-day moving averages."""
import random,json,math
from pathlib import Path
from time import perf_counter
def naive(values,k=7):
    """Return averages and explicit additions, using one slice per window."""
    if k<=0:raise ValueError('Positive window required')
    result=[];adds=0
    for start in range(len(values)-k+1):
        total=0
        for value in values[start:start+k]:total+=value;adds+=1
        result.append(total/k)
    return result,adds
def running(values,k=7):
    """Update one total per shifted window; count additions and subtractions."""
    if k<=0:raise ValueError('Positive window required')
    if len(values)<k:return [],0,0
    total=0;adds=0;subs=0
    for value in values[:k]:total+=value;adds+=1
    result=[total/k]
    for i in range(k,len(values)):
        total-=values[i-k];subs+=1
        total+=values[i];adds+=1
        result.append(total/k)
    return result,adds,subs
def weekly(values):
    """Include a final partial week; ties are reported at every position."""
    totals=[sum(values[i:i+7]) for i in range(0,len(values),7)]
    if not totals:return {'totals':[],'wettest':[],'driest':[],'dry_days':[]}
    highest,lowest=max(totals),min(totals)
    return dict(totals=totals,wettest=[i+1 for i,v in enumerate(totals) if v==highest],
                driest=[i+1 for i,v in enumerate(totals) if v==lowest],
                dry_days=[i+1 for i,v in enumerate(values) if v==0])
def deque_average(values,k=7):
    """Streaming alternative retaining at most k readings."""
    from collections import deque
    window=deque(maxlen=k);total=0;result=[]
    for value in values:
        if len(window)==k:total-=window[0]
        window.append(value);total+=value
        if len(window)==k:result.append(total/k)
    return result
if __name__=='__main__':
    sample=[0,1,2,3,4,5,6,7,0,0,0,0,0,0]
    assert weekly(sample)['totals']==[21,7]
    assert weekly(sample)['wettest']==[1] and weekly(sample)['driest']==[2]
    assert running(sample)[0]==naive(sample)[0]==deque_average(sample)
    for values in [[],[1,2,3],[0]*7]:assert
        naive(values)[0]==running(values)[0]==deque_average(values)
    rows=[]
    for n in [365,36500,365000]:
        rng=random.Random(1911260319)
        # Integer millimetres deliberately make running sums exact.
        data=[rng.randrange(0,101) if rng.random()>.2 else 0 for _ in range(n)]
        begin=perf_counter();a,adds1=naive(data);t1=perf_counter()-begin
        begin=perf_counter();b,adds2,subs=running(data);t2=perf_counter()-begin
        assert a==b and adds1==7*(n-6) and adds2==n and subs==n-7
        rows.append(dict(n=n,windows=len(a),naive_additions=adds1,running_additions=adds2,
                         running_subtractions=subs,naive_seconds=t1,running_seconds=t2))
        if n==365:
            summary=weekly(data)
            Path(__file__).with_name('year_summary.json').write_text(json.dumps(summary,indent=2))
            print('365-day weekly totals:',summary['totals'])
            print('Wettest weeks:',summary['wettest'],'driest weeks:',summary['driest'])
            print('Dry days:',summary['dry_days'])
    Path(__file__).with_name('measurements.json').write_text(json.dumps(rows,indent=2))
    print('Hand sample weekly totals [21, 7], moving averages, empty/short inputs PASS')
    print(json.dumps(rows,indent=2))
```

Q2/complete_code.py

```
rows, cols = 3, 4
bad = [[0] * cols] * rows
bad[0][1] = 7
good = [[0]*cols for _ in range(rows)]
good[0][1] = 7
print("bad :", bad)
print("good:", good)
print(bad[0] is bad[1], good[0] is good[1])
stops = {(13.0827, 80.2707): "Chennai Central", (13.0418, 80.2341): "T. Nagar"}
print(stops[(13.0418, 80.2341)])
```

Q2/copying.py

```
"""Aliasing, nested copies, tuple keys and equivalent size/timing experiments."""
import copy,sys,json,random
from pathlib import Path
from time import perf_counter
from statistics import mean
def mutate(values):
    """Mutate the same object that the caller passed."""
    values.append(9)
if __name__=='__main__':
    bad=[[0]*3]*3;good=[[0]*3 for _ in range(3)]
    print('bad row ids:',[id(row) for row in bad])
    print('good row ids:',[id(row) for row in good])
    assert bad[0] is bad[1] and good[0] is not good[1]
    original=[[1],[2]];alias=original;sliced=original[:];shallow=copy.copy(original);deep=copy.deep
        copy(original)
    original[0].append(7)
    assert alias is original and sliced is not original and shallow is not original
    assert sliced[0] is original[0] and shallow[0] is original[0] and deep==[[1],[2]]
    print('After nested edit:
        original=',original,'alias=',alias,'slice=',sliced,'shallow=',shallow,'deep=',deep)
    coordinates={(13.0827,80.2707):'Central',(13.0418,80.2341):'T. Nagar'}
    for (latitude,longitude),name in coordinates.items():print(name,latitude,longitude)
    try:coordinates[[13.0418,80.2341]]='invalid'
    except TypeError as error:print('List key rejected:',error)
    else:raise AssertionError('List key accepted')
    inside=([1,2],'stop');inside[0].append(3);assert inside==([1,2,3],'stop')
    try:hash(inside)
    except TypeError:print('Tuple containing a list is also unhashable')
    else:raise AssertionError('Mutable tuple member hashable')
    caller=[];mutate(caller);assert caller==[9]
    n=1000000;lists=[];tuples=[]
    for _ in range(3):
        begin=perf_counter();a=list(range(n));lists.append(perf_counter()-begin)
        begin=perf_counter();b=tuple(range(n));tuples.append(perf_counter()-begin)
    assert a==list(b)
    positions=[random.Random(i).randrange(n) for i in range(10000)]
    access={}
    for name,values in [('list',a),('tuple',b)]:
        times=[]
        for _ in range(3):
            begin=perf_counter();total=sum(values[i] for i in
                positions);times.append(perf_counter()-begin)
        access[name]={'runs_seconds':times,'mean_seconds':mean(times),'checksum':total}
    assert access['list']['checksum']==access['tuple']['checksum']
    result=dict(n=n,list_shallow_bytes=sys.getsizeof(a),tuple_shallow_bytes=sys.getsizeof(b),
                list_creation_runs=lists,tuple_creation_runs=tuples,indexing=access)
    Path(__file__).with_name('measurements.json').write_text(json.dumps(result,indent=2))
    print(json.dumps(result,indent=2))
```

Q3/complaints.py

```
"""Normalize words, count by get/setdefault/Counter and group complaints."""
from collections import Counter
from pathlib import Path
from time import perf_counter
import json
STOP={'the','not','and','is','high'}
def words(text):
    """Lowercase and turn punctuation/nonletters into word separators."""
    return ''.join(c.lower() if c.isalpha() else ' ' for c in text).split()
def dict_get(tokens):
    """Count each supplied token with dict.get."""
    result={}
    for word in tokens:result[word]=result.get(word,0)+1
    return result
def dict_setdefault(tokens):
    """Count using setdefault; grouping below also demonstrates its natural use."""
    result={}
    for word in tokens:result.setdefault(word,0);result[word]+=1
    return result
def list_count(tokens):
    """Scan the token list once per distinct word."""
    return {word:tokens.count(word) for word in set(tokens)}
def group(records):
    """Return district and crop dictionaries of original complaint records."""
    district={};crop={}
    for record in records:
        district.setdefault(record['district'],[]).append(record)
        crop.setdefault(record['crop'],[]).append(record)
    return district,crop
if __name__=='__main__':
    sample=[dict(district='Chennai',crop='rice',text='SEEDS, seeds! delayed.'),
            dict(district='Madurai',crop='rice',text='Price is HIGH; seeds.'),
            dict(district='Chennai',crop='millet',text='')]
    district,crop=group(sample)
    assert {key:len(value) for key,value in district.items()}=={'Chennai':2,'Madurai':1}
    assert len(crop['rice'])==2
    tokens=[w for r in sample for w in words(r['text']) if w not in STOP]
    assert
        dict_get(tokens)==dict_setdefault(tokens)==dict(Counter(tokens))=={'seeds':3,'delayed':1,'p
        rice':1}
    assert dict_get([])==list_count([])=={}
    print('District counts:',{k:len(v) for k,v in district.items()})
    print('Crop groups:',{k:len(v) for k,v in crop.items()})
    print('Case/punctuation/empty checks PASS:',dict_get(tokens))
    templates=['seeds not received seeds delayed','fertiliser price high',
               'seeds price high subsidy not received','water pump broken crop drying']
    results=[]
    for n in [1000,10000,100000]:
        texts=[templates[i%len(templates)] for i in range(n)]
        tokens=[w for text in texts for w in words(text) if w not in STOP]
        row={'complaints':n,'words':len(tokens),'distinct':len(set(tokens)),'seconds':{}}
        answers=[]
        for fn in [dict_get,dict_setdefault,list_count,Counter]:
            begin=perf_counter();answer=dict(fn(tokens));row['seconds'][fn.__name__]=perf_counter()
                -begin;answers.append(answer)
        assert all(answer==answers[0] for answer in answers)
        row['list_count_comparisons_model']=len(tokens)*len(set(tokens))
        results.append(row)
    Path(__file__).with_name('measurements.json').write_text(json.dumps(results,indent=2))
    print(json.dumps(results,indent=2))
```

Q3/complete_code.py

```
calls = [
    "seeds not received seeds delayed",
    "fertiliser price high",
    "seeds price high subsidy not received",
]
STOP = {"not", "high"}
def frequencies(texts, stop):
    counts={}
    for text in texts:
        for word in text.split():
            if word not in stop:counts[word]=counts.get(word,0)+1
    return counts
def top(counts, k):
    return sorted(counts.items(),key=lambda pair:(-pair[1],pair[0]))[:k]
print(top(frequencies(calls, STOP), 3))
```

Q4/complete_code.py

```
ai_elective = {"192311001", "192311004", "192311007", "192311009"}
iot_elective = {"192311002", "192311004", "192311009", "192311010"}
all_students = {f"1923110{i:02d}" for i in range(1, 11)}
both=ai_elective & iot_elective
exactly_one=ai_elective ^ iot_elective
neither=all_students-(ai_elective | iot_elective)
print("clash      :", sorted(both))
print("exactly one:", sorted(exactly_one))
print("neither    :", sorted(neither))
```

Q4/electives.py

```
"""Set operations, membership timing and equivalent intersection algorithms."""
import json,sys
from pathlib import Path
from time import perf_counter
def nested(a,b):
    """Compare every pair, returning distinct common values and comparison count."""
    result=set();comparisons=0
    for x in a:
        for y in b:
            comparisons+=1
            if x==y:result.add(x)
    return result,comparisons
def sorted_walk(a,b):
    """Sort and advance pointers through both lists."""
    a=sorted(a);b=sorted(b);i=j=0;result=set()
    while i<len(a) and j<len(b):
        if a[i]==b[j]:result.add(a[i]);i+=1;j+=1
        elif a[i]<b[j]:i+=1
        else:j+=1
    return result
if __name__=='__main__':
    a={'192311001','192311004','192311007','192311009'}
    b={'192311002','192311004','192311009','192311010'}
    universe={f'1923110{i:02d}' for i in range(1,11)}
    assert a.intersection(b)==a&b
    assert a.symmetric_difference(b)==a^b
    assert universe.difference(a.union(b))==universe-(a|b)
    assert a&b=={'192311004','192311009'}
    for x,y in [(set(),set()),(a,a),(a,set())]:
        assert nested(list(x),list(y))[0]==x&y==sorted_walk(list(x),list(y))
    assert set(['x','x','y'])=={'x','y'}
    print('Operator/method equality, hand case, duplicates, empty and identical sets PASS')
    membership=[]
    for n in [10000,100000,1000000]:
        values=[f'{i:010d}' for i in range(n)];lookup=set(values)
        probes=[values[-1],'missing']*10;row={'n':n,'probes':len(probes)}
        checks=[]
        for label,container in [('list',values),('set',lookup)]:
            begin=perf_counter();answers=[value in container for value in
                probes];row[label+'_seconds']=perf_counter()-begin;checks.append(answers)
        assert checks[0]==checks[1]
        row['set_shallow_bytes']=sys.getsizeof(lookup);membership.append(row)
    common=[]
    for n in [100,1000,3000]:
        a=list(range(n));b=list(range(n//2,n+n//2));row={'n':n,'m':len(b)}
        begin=perf_counter();old,count=nested(a,b);row['nested_seconds']=perf_counter()-begin
        begin=perf_counter();new=set(a)&set(b);row['set_seconds']=perf_counter()-begin
        begin=perf_counter();third=sorted_walk(a,b);row['sort_walk_seconds']=perf_counter()-begin
        assert old==new==third
        row['comparisons']=count;row['common']=len(new);common.append(row)
    data={'membership':membership,'intersection':common}
    Path(__file__).with_name('measurements.json').write_text(json.dumps(data,indent=2))
    print(json.dumps(data,indent=2))
```

Q5/sensors.py

```
"""Equivalent temperature cleaning pipelines retaining original minute indices."""
import random,sys,json
from pathlib import Path
from time import perf_counter
def loop(readings):
    """Filter, convert and collect alarm minutes with explicit append operations."""
    c=[];f=[];alarms=[]
    for minute,value in enumerate(readings,1):
        if -50<=value<=50:
            c.append(value);f.append(value*9/5+32)
            if not 2<=value<=8:alarms.append(minute)
    return c,f,alarms
def comprehension(readings):
    """Use list comprehensions and enumerate over the original sequence."""
    c=[value for value in readings if -50<=value<=50]
    f=[value*9/5+32 for value in c]
    alarms=[minute for minute,value in enumerate(readings,1) if -50<=value<=50 and not 2<=value<=8]
    return c,f,alarms
def functional(readings):
    """Use filter/map with lambdas and preserve original indices for alarms."""
    c=list(filter(lambda value:-50<=value<=50,readings))
    f=list(map(lambda value:value*9/5+32,c))
    alarms=list(map(lambda pair:pair[0],filter(lambda pair:-50<=pair[1]<=50 and not
        2<=pair[1]<=8,enumerate(readings,1))))
    return c,f,alarms
if __name__=='__main__':
    sample=[-51,-50,2,8,9,50,51]
    expected=([-50,2,8,9,50],[-58.0,35.6,46.4,48.2,122.0],[2,5,6])
    for fn in [loop,comprehension,functional]:assert fn(sample)==expected
    for data in [[],[2,3,8],[-50,50],[-51,51,100]]:
        assert loop(data)==comprehension(data)==functional(data)
    indexed={i:c for i,c in enumerate(sample,1) if -50<=c<=50}
    distinct={c for c in sample if -50<=c<=50}
    assert indexed=={2:-50,3:2,4:8,5:9,6:50}
    print('Sample Celsius/Fahrenheit/alarm minutes:',expected)
    print('Dictionary comprehension:',indexed,'set comprehension:',sorted(distinct))
    print('Empty/all-valid/all-invalid/boundary tests PASS')
    rows=[]
    for n in [10000,100000,1000000]:
        rng=random.Random(1911260319);values=[rng.randint(-60,60) for _ in range(n)]
        row={'n':n,'seconds':{}};reference=None
        for fn in [loop,comprehension,functional]:
            begin=perf_counter();actual=fn(values);row['seconds'][fn.__name__]=perf_counter()-begin
            if reference is None:reference=actual
            else:assert actual==reference
        materialized=[c for c in values if -50<=c<=50]
        generator=(c for c in values if -50<=c<=50)
        row['valid_count']=len(materialized)
        row['list_shallow_bytes']=sys.getsizeof(materialized);row['generator_shallow_bytes']=sys.ge
            tsizeof(generator)
        begin=perf_counter();a=sum(materialized);row['sum_list_seconds']=perf_counter()-begin
        begin=perf_counter();b=sum(generator);row['sum_generator_seconds']=perf_counter()-begin
        assert a==b
        rows.append(row)
    Path(__file__).with_name('measurements.json').write_text(json.dumps(rows,indent=2))
    print(json.dumps(rows,indent=2))
```

Q6/timetable.py

```
"""Fictional same-day timetables with a station-to-stop index."""
import json,sys
from pathlib import Path
from time import perf_counter
DAYS={'Mon','Tue','Wed','Thu','Fri','Sat','Sun'}
def fixture():
    """Five illustrative trains, not a real railway service schedule."""
    return {
      'T101':dict(name='Village Link',stops=[('North','08:00','08:05'),('Central','09:00','09:05'),
          ('South','10:00','10:05')],days={'Mon','Wed','Fri'}),
      'T102':dict(name='City Connector',stops=[('East','07:00','07:05'),('Central','08:30','08:35')
          ,('West','10:00','10:05')],days=set(DAYS)),
      'T103':dict(name='Coastal Local',stops=[('Central','11:00','11:05'),('Beach','12:00','12:05')
          ,('Port','13:00','13:05')],days={'Sat','Sun'}),
      'T104':dict(name='Valley Link',stops=[('Hill','06:00','06:05'),('Central','07:00','07:05'),('
          Lake','08:00','08:05')],days={'Mon','Tue'}),
      'T105':dict(name='Market Local',stops=[('North','12:00','12:05'),('Market','13:00','13:05'),(
          'South','14:00','14:05')],days=set(DAYS)),
    }
def scan(data,station,day):
    """Visit all stops, retain running-day matches and sort by departure."""
    result=[]
    for train,info in data.items():
        for stop,arr,dep in info['stops']:
            if stop==station and day in info['days']:result.append((train,info['name'],arr,dep))
    return sorted(result,key=lambda row:(row[3],row[0]))
def build_index(data):
    """Index every occurrence directly to avoid rescanning a train's stops."""
    index={}
    for train,info in data.items():
        for station,arr,dep in info['stops']:index.setdefault(station,[]).append((train,arr,dep))
    return index
def indexed(data,index,station,day):
    """Filter only stop occurrences at the queried station, then sort."""
    matches=[(train,data[train]['name'],arr,dep) for train,arr,dep in index.get(station,[]) if day
        in data[train]['days']]
    return sorted(matches,key=lambda row:(row[3],row[0]))
def flat_scan(rows,station,day):
    """Flat-list alternative with the same query output."""
    result=[(train,name,arr,dep) for train,name,stop,arr,dep,days in rows if stop==station and day
        in days]
    return sorted(result,key=lambda row:(row[3],row[0]))
def generated(n):
    """Build n trains with ten unique stations per train and zero-padded times."""
    result={}
    for i in range(n):
        stops=[]
        for j in range(10):
            minute=360+j*45+i%30
            stops.append((f'S{(i+j)%100:03d}',f'{minute//60:02d}:{minute%60:02d}',f'{(minute+5)//60
                :02d}:{(minute+5)%60:02d}'))
        result[f'T{i:05d}']=dict(name=f'Train {i}',stops=stops,days={'Mon','Wed','Fri'} if i%2 else
             set(DAYS))
    return result
if __name__=='__main__':
    data=fixture();index=build_index(data)
    expected=['T104','T102','T101']
    actual=scan(data,'Central','Mon');assert [r[0] for r in actual]==expected
    assert actual==indexed(data,index,'Central','Mon')
    assert scan(data,'Unknown','Mon')==[]==indexed(data,index,'Unknown','Mon')
    assert all(row[0]!='T103' for row in actual)
    print('Fictional timetable: Central on Monday')
    for train,name,arrival,departure in
        actual:print(f'{train:<7}{name:<18}{arrival:>7}{departure:>7}')
    print('Third stop departure:',data['T101']['stops'][2][2])
    print('Unknown station and non-running-day checks PASS')
    results=[]
    for n in [100,1000,10000]:

            data=generated(n);begin=perf_counter();index=build_index(data);build_time=perf_counter(
            )-begin
        flat=[(train,info['name'],station,arr,dep,info['days']) for train,info in data.items() for
            station,arr,dep in info['stops']]
        row={'trains':n,'total_stops':n*10,'station_candidates':len(index.get('S005',[])),'queries'
            :20,'index_build_seconds':build_time}
        outputs=[]
        for name,fn in [('scan',lambda:scan(data,'S005','Mon')),('indexed',lambda:indexed(data,inde
            x,'S005','Mon')),('flat',lambda:flat_scan(flat,'S005','Mon'))]:
```

```
        begin=perf_counter()
        for _ in range(20):answer=fn()
        row[name+'_seconds']=(perf_counter()-begin)/20;outputs.append(answer)
    assert outputs[0]==outputs[1]==outputs[2]
    # Count newly allocated container overhead, excluding referenced strings/base data.
    row['index_container_bytes']=sys.getsizeof(index)+sum(sys.getsizeof(v)+sum(sys.getsizeof(t)
         for t in v) for v in index.values())
    results.append(row)
Path(__file__).with_name('measurements.json').write_text(json.dumps(results,indent=2))
print(json.dumps(results,indent=2))
```

Q7/contacts.py

```
"""Merge connected contacts using shared canonical phone numbers, not names."""
import re,json
from pathlib import Path
from time import perf_counter
def phone(raw):
    """Assume Indian 10-digit numbers; accept +91/0091 and benign separators."""
    text=str(raw).strip()
    if any(c not in '0123456789+ -().' for c in text):return None
    if text.count('+')>1 or ('+' in text and not text.startswith('+91')):return None
    digits=''.join(c for c in text if c in '0123456789')
    if digits.startswith('0091') and len(digits)==14:digits=digits[4:]
    elif digits.startswith('91') and len(digits)==12:digits=digits[2:]
    if len(digits)!=10:return None
    return '+91'+digits
def name(raw):
    """Normalize display aliases; names alone never imply identity."""
    parts=re.sub(r'[^\w\s]',' ',str(raw or '').casefold()).split()
    if parts and parts[0] in {'dr','mr','mrs','ms'}:parts=parts[1:]
    return ' '.join(parts)
def prepare(records):
    """Canonicalize fields; invalid phones are omitted and flagged."""
    result=[]
    for i,record in enumerate(records):
        if not isinstance(record,dict):raise ValueError('Contact must be a dictionary')
        numbers=set();invalid=[]
        for raw in record.get('numbers',[]) or []:
            normalized=phone(raw)
            if normalized:numbers.add(normalized)
            else:invalid.append(str(raw))
        emails={str(e).strip().casefold() for e in (record.get('emails',[]) or []) if
            str(e).strip()}
        alias=name(record.get('name',''))
        result.append(dict(names={alias} if alias else
            set(),numbers=numbers,emails=emails,invalid_numbers=invalid,source_ids={i}))
    return result
def combine(prepared,root):
    """Gather connected records into deterministic, JSON-friendly reports."""
    groups={}
    for i,record in enumerate(prepared):
        group=groups.setdefault(root(i),dict(names=set(),numbers=set(),emails=set(),source_ids=set(
            ),invalid_numbers=[]))
        for field in ['names','numbers','emails','source_ids']:group[field].update(record[field])
        group['invalid_numbers'].extend(record['invalid_numbers'])
    output=[]
    for group in groups.values():output.append({key:sorted(value) for key,value in group.items()})
    return output  # Groups are first encountered in original input order.
def structure(n):
    """Union-find with path compression and union by size handles bridge records."""
    parents=list(range(n));sizes=[1]*n
    def find(i):
        """Return root and compress paths."""
        while parents[i]!=i:
            parents[i]=parents[parents[i]];i=parents[i]
        return i
    def union(a,b):
        """Join components, attaching the smaller tree to the larger."""
        a,b=find(a),find(b)
        if a==b:return
        if sizes[a]<sizes[b]:a,b=b,a
        parents[b]=a;sizes[a]+=sizes[b]
    return find,union
def merge(records):
    """Dictionary number -> representative; union every shared-number edge."""
    prepared=prepare(records);find,union=structure(len(prepared));owner={};lookups=0
    for i,record in enumerate(prepared):
        for number in record['numbers']:
            lookups+=1
            if number in owner:union(i,owner[number])
            else:owner[number]=i
    return combine(prepared,find),lookups
def pairwise(records,checkpoint=None):
    """Every-pair reference; optional progress callback counts attempted comparisons."""
    prepared=prepare(records);find,union=structure(len(prepared));comparisons=0
    for i in range(len(prepared)):
        for j in range(i):
            comparisons+=1
            if prepared[i]['numbers'] & prepared[j]['numbers']:union(i,j)
            if checkpoint and comparisons%100000==0:checkpoint(comparisons)
```

```
    return combine(prepared,find),comparisons
def generated(n):
    """Five formatted records per person, with a bounded phone field size."""
    records=[]
    for i in range(n):
        person=i//5;digits=f'{9000000000+person:010d}'
        written='+91 '+digits[:5]+' '+digits[5:] if i%2 else digits
        records.append(dict(name=f'Person
            {person}',numbers=[written],emails=[f'p{person}@example.test']))
    return records
def fixture():
    """Synthetic examples include a bridge, same names and missing fields."""
    return [dict(name='Dr. Ravi',numbers=['+91 98400 12345'],emails=['RAVI@example.test']),
            dict(name='ravi
                kumar',numbers=['9840012345','9876543210'],emails=['ravi.work@example.test']),
            dict(name='R. Kumar',numbers=['0091 9876543210'],emails=[]),
            dict(name='ravi kumar',numbers=['9123456789'],emails=[]),
            dict(name='No Phone',emails=['solo@example.test']),
            dict(name='',numbers=['invalid'])]
if __name__=='__main__':
    sample=fixture();actual,lookups=merge(sample);reference,comparisons=pairwise(sample)
    assert actual==reference and len(actual)==4
    assert actual[0]['source_ids']==[0,1,2] and len(actual[0]['numbers'])==2 and
        len(actual[0]['emails'])==2
    assert actual[1]['source_ids']==[3] and actual[2]['source_ids']==[4] and
        actual[3]['source_ids']==[5]
    assert merge([])==([],0)
    assert phone('+91 98400 12345')==phone('0091 9840012345')=='+919840012345'
    bridge=[{'numbers':['9000000001']},{'numbers':['9000000002']},{'numbers':['9000000001','9000000
        002']}]
    assert len(merge(bridge)[0])==1 and merge(bridge)[0]==pairwise(bridge)[0]
    print('Hand fixture groups, aliases, same-name separation, bridge and empty tests PASS')
    print(json.dumps(actual,indent=2))
    Path(__file__).with_name('sample_input.json').write_text(json.dumps(sample,indent=2))
    Path(__file__).with_name('sample_output.json').write_text(json.dumps(actual,indent=2))
    rows=[]
    for n in [1000,100000]:

            records=generated(n);begin=perf_counter();groups,count=merge(records);elapsed=perf_coun
            ter()-begin
        assert len(groups)==n//5 and sum(len(g['source_ids']) for g in groups)==n
        row=dict(n=n,groups=len(groups),dictionary_seconds=elapsed,phone_occurrences=count,required
            _pair_comparisons=n*(n-1)//2)
        if n==1000:

                begin=perf_counter();other,comp=pairwise(records);row['pairwise_seconds']=perf_coun
                ter()-begin
            assert other==groups and comp==n*(n-1)//2
        rows.append(row)
    Path(__file__).with_name('measurements.json').write_text(json.dumps(rows,indent=2))
    print(json.dumps(rows,indent=2))
```

Q7/large_comparison.py

```
"""Attempt the quadratic 100000-record alternative with a three-second watchdog."""
import subprocess,sys,tempfile,json
from pathlib import Path
from time import perf_counter
if __name__=='__main__':
    with tempfile.TemporaryDirectory() as directory:
        path=Path(directory)/'progress.json';begin=perf_counter()
        try:
            run=subprocess.run([sys.executable,str(Path(__file__).with_name('pairwise_trial.py')),s
                tr(path)],capture_output=True,text=True,timeout=3,check=True)
            result={'status':'COMPLETED','result':json.loads(run.stdout)}
        except subprocess.TimeoutExpired:
            result={'status':'TIMEOUT_3_SECONDS','parent_elapsed_seconds':perf_counter()-begin,
                    'progress_checkpoint':json.loads(path.read_text()) if path.exists() else None,
                    'required_pairs':4999950000,'note':'No full-run completion time is claimed.'}
    Path(__file__).with_name('large_trial.json').write_text(json.dumps(result,indent=2))
    print(json.dumps(result,indent=2))
```

Q7/pairwise_trial.py

```
"""Bounded every-pair 100000-contact trial; progress is never fabricated."""
from contacts import generated,pairwise
from pathlib import Path
from time import perf_counter
import sys,json
progress=Path(sys.argv[1]);records=generated(100000);begin=perf_counter()
def save(count):
    """Save actual progress periodically, outside any inference about completion."""
    progress.write_text(json.dumps({'completed_pair_comparisons':count,'worker_elapsed_seconds':per
        f_counter()-begin}))
answer,count=pairwise(records,save)
print(json.dumps({'groups':len(answer),'comparisons':count,'seconds':perf_counter()-begin}))
```

tools/run_all.py

```
"""Run all CO3 experiments and preserve genuine output and execution metadata."""
from pathlib import Path
import subprocess,sys,json

ROOT=Path(__file__).resolve().parents[1]
JOBS={'Q1':['rainfall.py'],'Q2':['complete_code.py','copying.py'],
      'Q3':['complete_code.py','complaints.py'],'Q4':['complete_code.py','electives.py'],
      'Q5':['sensors.py'],'Q6':['timetable.py'],
      'Q7':['contacts.py','large_comparison.py']}

def main():
    """Run every example with assertions enabled; stop if any program fails."""
    results=[]
    for folder,files in JOBS.items():
        output=[]
        for file in files:
            command=f'{folder}/{file}'
            run=subprocess.run([sys.executable,command],cwd=ROOT,text=True,
                               capture_output=True,check=True,timeout=90)
            output.append(f'$ python3 {command}\n'+run.stdout)
            if run.stderr:output.append('STDERR:\n'+run.stderr)
            results.append({'program':command,'exit_code':run.returncode})
        (ROOT/folder/'output.txt').write_text('\n'.join(output))
        print(folder,'PASS',flush=True)
    (ROOT/'verification.json').write_text(json.dumps({'python':sys.version,'results':results},inden
        t=2)+'\n')
    print('All programs passed. The intentionally bounded Q7 trial may report TIMEOUT_3_SECONDS.')

if __name__=='__main__':main()
```